DYNAMIC HIDDEN STATE HEAP FOLDING FOR EFFICIENT ATTENTION (HFOLD)

Aditya Dewan (adewan2) [*1] **Pranav Ram (pram)** [*1] **Krish Mody (kmody)** [*1]

ABSTRACT

Full self-attention provides strong long-context reasoning but scales quadratically with sequence length, making long-context inference expensive. We introduce HFold, an inference-time heap-folding mechanism that augments sliding-window attention with a bounded cache of high-relevance hidden states. HFold retrieves selected historical states during inference and folds evicted states back into the heap using learned summary and relevance models. We evaluate HFold against full attention and sliding-window attention on Pythia 14M and 31M using WikiText and SCROLLS. Our results suggest that HFold can recover global token relationships lost by sliding-window attention more effectively, but the current prototype is not hardware optimized and has substantial throughput overhead.

1 INTRODUCTION

Attention, particularly self-attention, has become a hallmark of modern deep learning. It allows token representations to weigh themselves against other transformed representations according to relevance. This is especially useful for sequence modeling and language tasks, where each token can condition on other tokens to learn higher-order relationships in text and other modalities. Attention is therefore a central component of the Transformer architecture (Vaswani et al., 2017) and its descendants, including GPT, Qwen, and Llama-style models.

The efficiency of attention directly affects the deployment, feasibility, and scalability of modern large language models and Transformers more generally. Full self-attention is expensive for long sequences because it requires all-pairs token interactions. Improving attention efficiency while preserving long-context behavior is therefore important for practical low-compute and long-context inference.

This project studies Dynamic Hidden State Heap Folding for Efficient Attention, or HFold. Our main contributions are:

- A heap-based augmentation of sliding-window attention that preserves constant-size attention computation per timestep while incorporating compressed historical information.
- A recursive hidden-state folding mechanism that conditions retained historical keys on evicted or previously attended tokens.
- An empirical comparison against full attention, standard sliding-window attention, and selected long-context baselines on long-sequence language modeling and benchmark tasks.

2 PROBLEM

Research questions. This project is organized around the following research questions:

- Can sliding-window attention recover some long-range recall ability by maintaining a compact heap of historically high-attention hidden states?
- Can the proposed heap folding mechanism preserve the linear-time behavior of local attention while improving long-context perplexity or recall-sensitive performance?
- How do the compute and memory tradeoffs of HFold compare against full attention, standard sliding-window attention, and sparse long-context methods?

Attention is a key component of modern deep learning and Transformer architectures, but full self-attention is limited by its all-pairs dot-product computation. This operation has quadratic time and memory complexity in the sequence length. Long-context models therefore require approximations that reduce computation while retaining enough long-range information to perform recall, conditioning, and generation tasks effectively.

Several approaches approximate attention to improve scalability. Sliding-window attention restricts each token to a fixed local context of width w, such as the previous $\frac{w}{2} + 1$

tokens in autoregressive generation (Beltagy et al., 2020). Other methods approximate softmax attention using linear maps (Katharopoulos et al., 2020), while State Space Model approaches provide alternative sequence modeling mechanisms (Gu et al., 2022). These methods aim for $O(n)$-style sequence modeling, but local attention can degrade direct token-recall performance and weaken long-range conditioning as sequence length increases.

3 Related Work

A major line of work on efficient long-context Transformers replaces dense all-to-all attention with structured sparse attention patterns. Longformer (Beltagy et al., 2020) is one representative example. It replaces standard self-attention with a combination of local sliding-window attention and a small number of task-specific global attention tokens. The sliding-window pattern allows each token to attend to nearby tokens, which preserves local contextual modeling while reducing the cost of attention from quadratic to linear in sequence length. The global tokens provide a limited mechanism for nonlocal information flow, which is useful for document-level tasks such as question answering or classification.

However, Longformer's long-range connectivity is largely determined by a fixed architectural pattern or by task-specified global tokens. This means that tokens outside the local window are not generally available unless they are connected through the predefined sparse structure. HFold is motivated by this limitation: rather than relying only on fixed global positions, it dynamically selects high-relevance historical states and stores them in a bounded heap for future reuse.

BigBird (Zaheer et al., 2020) extends sparse attention by combining local window attention, random attention, and global attention. The local component captures nearby context, the random component improves graph connectivity across distant tokens, and the global component allows selected tokens to attend broadly across the sequence. This sparse attention pattern reduces the quadratic dependency of full attention to linear complexity and provides theoretical guarantees, including universal approximation and Turing-completeness properties under the proposed sparse structure.

Despite these advantages, BigBird's nonlocal connections are still mostly fixed or randomized rather than chosen based on the actual relevance of previous hidden states during inference. HFold differs by making the retained long-range context content-dependent: hidden states are inserted into the heap based on attention scores, and the highest-ranked states are retrieved later. In this sense, HFold keeps the linear-time motivation of sparse attention while replacing random or manually specified nonlocal access with a dynamic memory mechanism.

Routing Transformer (Roy et al., 2021) takes a more adaptive approach to sparse attention. Instead of using only fixed local or random sparse patterns, it uses content-based clustering so that tokens attend to other tokens assigned to related clusters. This lets the model allocate attention to tokens that are likely to be semantically or syntactically relevant, rather than forcing attention to follow only local positions. Routing Transformer combines this content-based routing with local attention and reduces the attention cost below the quadratic cost of full attention.

The main limitation for our setting is that routing introduces additional complexity. Cluster assignments must be computed dynamically, and the routing mechanism can add implementation overhead during training and inference. HFold shares the goal of content-dependent long-range access, but it uses a simpler bounded-memory interface. Instead of reclustering all tokens, HFold maintains a fixed-capacity heap of previously high-relevance states and retrieves only the top k states at each timestep. This gives HFold a constant-size augmentation to sliding-window attention while still allowing the chosen nonlocal states to depend on the model's own attention behavior.

A separate line of work studies how Transformers preserve and reuse intermediate representations. Attention Residuals (Kimi Team: Chen et al., 2026) is relevant because it rethinks the residual stream. Standard PreNorm Transformers accumulate layer outputs with fixed unit weights, which can dilute the contribution of individual layers as depth increases. Attention Residuals replaces this fixed accumulation with learned, input-dependent attention over previous layer representations, allowing each layer to selectively aggregate earlier hidden states.

HFold is related to this idea at a high level because both methods treat hidden states as reusable information rather than disposable intermediate activations. However, the two methods operate along different axes. Attention Residuals attends over representations across network depth, while HFold stores and reuses representations across sequence time. HFold also focuses specifically on long-context inference under a sliding-window attention constraint. Its heap is intended to preserve selected historical token states that would otherwise fall outside the local attention window.

Positioning of Hfold: HFold sits between fixed sparse attention and fully learned routing. Like Longformer and BigBird, it aims to preserve the efficiency benefits of sparse or local attention by ensuring that each timestep attends to only a fixed number of vectors. However, unlike these fixed sparse methods, HFold chooses its retained nonlocal states dynamically using attention scores. Like Routing

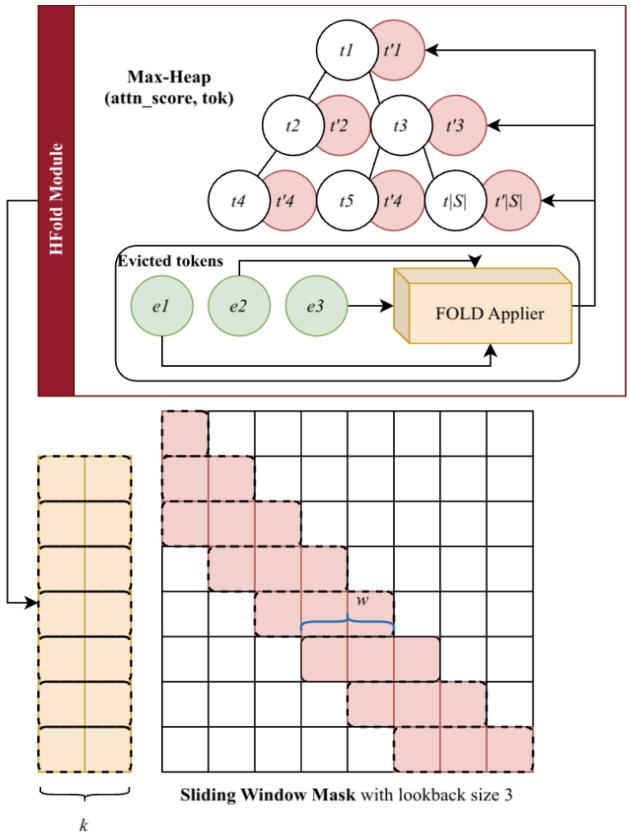


Figure 1. Overview of HFold. At each timestep, the model retrieves the top k states t_i from a bounded heap S and attends over them alongside the local context. The updated states t'_i are reinserted with the top w high-attention tokens from the current window, while evicted states e_i are folded back into the heap to preserve historical information.

Transformer, HFold introduces content-dependent access to nonlocal information, but it avoids full dynamic clustering by using a bounded heap. Finally, unlike residual-memory methods such as Attention Residuals, HFold uses historical token-level hidden states as an inference-time memory for long-context attention.

4 Overview

System overview. Figure 1 presents the full HFold mechanism in one diagram. HFold (Heap Folding) is a technique that introduces a lightweight, inference-time "ranked" fixed cache (a matrix heap) to store dynamically compressed representations of previously high-attention tokens outside the current window for autoregressive models. At each timestep, the model pops the top k vectors $\{t_1, t_2, \ldots, t_k\}$ from the heap S and concatenates them with the input at the current timestep q, allowing the model to attend over both local context and globally relevant historical tokens at inference time, while evaluating both pretrained and fine-tuned backbone checkpoints. These k vectors pass through all Transformer layers alongside the current input, producing updated representations $\{t'_1, t'_2, \ldots, t'_k\}$, which are then pushed back onto the heap alongside the top w highest-attention-scoring non-heap tokens from the current window. All evicted tokens $\{e_1, e_2, \ldots, e_m\}$ are then FOLDed back into the heap, augmenting each retained heap state with past context.

Project scope. For this course project, the implemented system can focus on the HFold attention mechanism itself rather than a full end-to-end model stack. The report should clearly distinguish the new HFold component from adopted model code, datasets, baselines, and training infrastructure.

5 Method

5.1 Overall HFold operation

HFold (Heap Folding) is a novel technique that introduces a lightweight, inference-time "ranked" fixed cache, implemented as a matrix heap, to store dynamically compressed representations of previously high-activation tokens present outside the current window for autoregressive models. The goal is to allow the model to attend over both local context and globally relevant historical tokens at inference time.

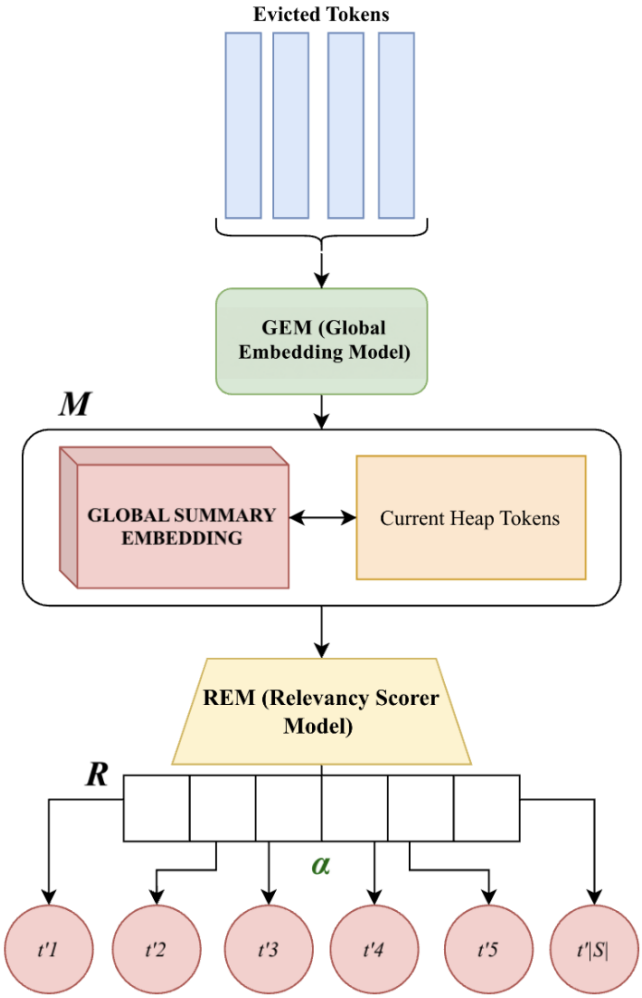


Figure 2. FOLD operation. Evicted vectors are summarized by GEM, concatenated with current heap states, and passed through RSM to produce relevance-conditioned updates for the active heap entries.

Let S denote the fixed-capacity heap. At each timestep, HFold pops the top k vectors from the heap S and concatenates them with the input at the current timestep q. These retrieved heap vectors augment the local sliding-window context, allowing the model to condition on compressed historical states that would otherwise fall outside the current attention window.

The retrieved k vectors pass through all Transformer layers alongside the current input. Their post-attention transformed representations are then pushed back onto the heap, alongside the top w highest-attention-scoring non-heap tokens from the current window. The heap therefore maintains a compact set of high-relevance historical states, where states that repeatedly receive high attention can remain available across future timesteps.

After computing attention over the local window and selected heap states, HFold identifies the top w non-heap vectors associated with the highest attention scores. These vectors are inserted into the heap, where the heap key is the attention score and the heap value is the corresponding transformed hidden representation. If inserting new vectors causes the heap to exceed its fixed capacity, lower-ranked or evicted states are not discarded directly. Instead, all evicted tokens are FOLDed back into the heap, augmenting each retained token with past context.

5.2 The FOLD operation

The FOLD operation is triggered when heap capacity exceeds $|S|$. FOLDing allows heap hidden states to accumulate relevant information from decayed or evicted states, minimizing lost data. Rather than dropping evicted tokens completely, HFold summarizes them and uses that summary to update the active heap states.

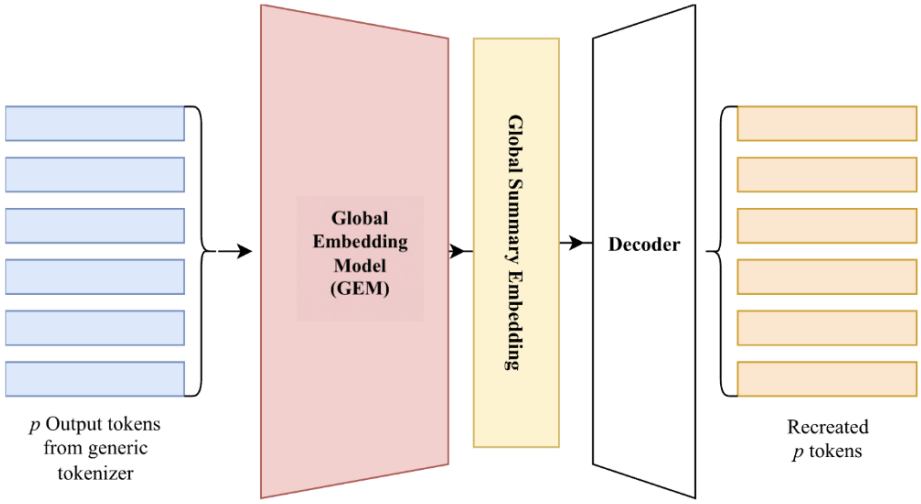


Figure 3. GEM and RSM modules. GEM compresses evicted tokens into a global summary embedding, while RSM estimates relevance scores used to update the active heap states.

Let $\{e_1, e_2, \ldots, e_m\}$ denote the collection of evicted vectors. Since HFold inserts the top w non-heap vectors at timestep t, the number of evicted tokens is

$$m = |e_t| = \max(0, |S_{t-1}| + w - |S|).$$

HFold first collects all evicted vectors $\{e_1, e_2, \ldots, e_m\}$, then passes them through a Global Embedding Model (GEM) to receive a summary eviction embedding e^*.

Next, HFold reads all compressed token vectors currently in S into a heap-state matrix T. It then builds an augmented input matrix

$$M = \text{concat}(e^*, T).$$

The augmented matrix M is passed into a Relevancy Scorer Model (RSM), which produces $|S|$ relevance scores in vector r. These scores determine how strongly the eviction summary should update each heap state. Each heap state is then updated according to

$$\forall t_i \in S, \quad t_i' = t_i + \alpha r_i e^*,$$

where t_i is the current heap state, r_i is the scalar relevance score assigned to that heap state, e^* is the eviction summary embedding, and α controls the update strength.

This turns each heap entry into a compressed hidden state that is conditioned on prior removed states. The intended effect is similar to recurrent state accumulation: a future token that attends to a heap entry can indirectly condition on a larger historical context than the explicit local window and retrieved heap states currently visible to attention.

5.3 Global Embedding Model

The Global Embedding Model (GEM) is used to compress evicted tokens into a global summary embedding before they are folded into the heap. The GEM autoencoder was trained to reconstruct token sequences produced by the GPT-NeoX tokenizer, using the compressed latent state as a global summary embedding for the input tokens.

In the HFold pipeline, GEM receives the evicted token vectors and produces the summary eviction embedding e^*. This embedding acts as a compact representation of information that would otherwise be removed from memory. By passing evicted vectors through GEM before updating the heap, HFold preserves information from decayed or evicted states while keeping the heap capacity fixed.

5.4 Relevancy Scoring Model

The Relevancy Scoring Model (RSM) is an encoder-only Transformer used to score the relevance of current heap states with respect to the global eviction summary. RSM was trained by taking the outputs of a generic LLM, Pythia in our experiments, and training an encoder-only Transformer to output a vector of the size of the heap at the CLS token. The objective is to mimic the attention scores generated by the output layer.

These attention scores are an in-built representation of token relevance with respect to other tokens, and therefore serve as an approximation for understanding how critical a compressed heap state vector is relative to the others. During FOLD, RSM receives the augmented matrix $M = \text{concat}(e^*, T)$ and outputs $|S|$ relevance scores r. HFold uses these scores to update the retained heap states,

allowing the heap to absorb information from evicted tokens in a relevance-aware manner.

5.5 Complexity analysis

Each attention computation views a fixed number of vectors: the local sliding-window context together with the top k retrieved heap states. Because the local window size, heap size, retrieval count, and insertion count are fixed hyperparameters, the number of vectors attended at each timestep remains constant.

Heap push and pop operations cost $O(\log |S|)$, but $|S|$ is fixed, so heap maintenance is $O(1)$ per timestep. Across an input sequence of length n, the total asymptotic cost remains $O(n)$, while still incorporating compressed information from tokens outside the current local window.

Although HFold is asymptotically linear, the current implementation is not hardware optimized. Heap operations introduce substantial overhead, so runtime results should be viewed as preliminary. Future optimization should replace explicit heap operations with a Top-K matrix heap and custom fused kernels to enable fair evaluation of HFold on latency, throughput, and memory use.

5.6 Implementation details

Implemented components. We implemented the HFold inference wrapper, bounded heap cache, heap retrieval and insertion logic, FOLD update, GEM module, RSM module, and evaluation code for running full-attention, sliding-window, and HFold inference modes. HFold is applied as an inference-time extension rather than a full retraining of the backbone model.

Adopted components. We adopted pretrained Pythia model checkpoints as the backbone language models. We also used existing tokenizer, dataloader, and baseline attention implementations where applicable. Full attention and sliding-window attention serve as the primary baselines.

Language and libraries. The implementation uses Python with PyTorch and Hugging Face Transformers, with experiments run on CUDA-enabled GPU hardware. The evaluation pipeline compares full-attention, sliding-window, and HFold inference under the same model and dataset settings.

6 Evaluation

We evaluate HFold against full attention and sliding-window attention on the Pythia 14M and 31M models (Biderman et al., 2023). The results reported below are averaged across both models; for SCROLLS, results are also averaged across tasks. We use WikiText (Merity et al., 2016) for language modeling and SCROLLS (Shaham et al., 2022) for long-context evaluation.

Relevant hyperparameters are as follows: A100 GPU; sliding-window size $W = 64$; heap size $|S| = 16$; retrieval count $k = 8$; insertion count $w = 8$; fine-tuned epochs: 2.

We report two settings: baseline, using pretrained models with no task fine-tuning, and fine-tuned, using the same backbones after 2 epochs. In both settings, we evaluate all three inference modes: full attention, sliding-window attention, and HFold. All methods use the same datasets and decoding/evaluation pipeline.

For language modeling, we report loss, perplexity, and tokens per second. For SCROLLS, we report mean task score, mean tokens per second, and mean examples per second. Lower loss and perplexity are better, while higher task score and throughput are better.

Table 1 shows the language modeling evaluation results. When the fine-tuned model is trained with full attention, HFold improves over sliding-window inference by reducing perplexity from 225.1 to 91.7. HFold does not match full attention, which obtains perplexity 62.2, but it recovers a substantial amount of quality lost by restricting the model to the local window.

When the fine-tuned model is trained with sliding-window attention, HFold remains close to the sliding-window baseline. Sliding-window inference obtains perplexity 91.7, while HFold obtains perplexity 93.0. This suggests that HFold transfers reasonably well to sliding-window fine-tuned checkpoints, even though these checkpoints are optimized for local sliding-window attention rather than HFold's heap-based memory.

Table 2 shows the SCROLLS mean results in the baseline setting. HFold is roughly competitive with sliding-window attention on aggregate task quality, obtaining a mean score of 3.20 compared to 3.21 for sliding-window attention. Full attention obtains the highest mean score at 3.58.

7 Conclusion

HFold is an inference-time heap-folding mechanism that augments sliding-window attention with a dynamic bounded heap cache of high-relevance historical hidden states. Across the evaluated Pythia 14M and 31M models, HFold improves over sliding-window attention in the language modeling setting when evaluated from a full-attention fine-tuned checkpoint, reducing perplexity from 225.1 to 91.7, while full attention achieves 62.2. On SCROLLS, HFold is roughly competitive with sliding-window attention in aggregate task quality, with a mean score of 3.20 compared to 3.21 for sliding-window attention and 3.58 for full attention. These results suggest that heap-based hidden-state

Table 1. LM evaluation results. Results are averaged across Pythia 14M and 31M.

Fine-tuned model	Eval mode	Loss ↓	PPL ↓	Tok/s ↑
Full attention	Full attention	4.13	62.2	59,993
Full attention	Sliding window	5.42	225.1	40,331
Full attention	HFold	4.52	91.7	96
Sliding window	Full attention	4.31	74.4	60,641
Sliding window	Sliding window	4.52	91.7	52,818
Sliding window	HFold	4.53	93.0	96

Table 2. SCROLLS mean results. Results are averaged across tasks in the baseline setting.

Eval mode	Mean score ↑	Mean tok/s ↑	Mean ex/s ↑
Full attention	3.58	1,702	1.57
Sliding window	3.21	1,475	1.36
HFold	3.20	80	0.07

w

reuse can recover some useful long-range context beyond the local window, but the current prototype has additional throughput overhead because its heap operations are not hardware optimized. Future work should focus on replacing explicit heap manipulation with Top-K matrix operations and fused kernels to evaluate HFold more fairly on latency, throughput, and memory efficiency.

REFERENCES

Beltagy et al. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

Katharopoulos et al. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, pp. 5156–5165. PMLR, 2020.

Kimi Team: Chen, G., Zhang, Y., Su, J., and Xu, e. a. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Roy et al. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, volume 9, pp. 53–68, 2021.

Shaham et al. SCROLLS: Standardized CompaRison over long language sequences. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 12007–12021, 2022.

Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Zaheer et al. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, volume 33, pp. 17283–17297, 2020.